

Latihan Soal Notasi Algoritmik & Alkyl

Nama : Muhammad Faran Aiki
NIM : 18225122
Kelas : 04
Jurusan : Sistem dan Teknologi Informasi - Jatinangor Reguler
Matkul : Algoritma dan Pemrograman I
Semester : 2

Komen

Ada dua jawaban, yang pertama menggunakan notasi algoritma, kedua **bahasa saya sendiri** di <https://github.com/FaranAiki/alkyl> (sbg tambahan) dengan commit hash ce6282ccce2e92ba731fb0dbba1b3922f3ea79bd (karena sekarang lagi mengubah dari AST → LLVM IR jadi AST → AST checked → ALIR → ALIR checked → LLVM IR agar lebih sesuai dengan matkul IF5120 Desain Kompilator).

Jawaban Notasi Algoritmik

1.

Program Fisika

```
{ Menghitung jarak }
```

Kamus

```
s, v, t: integer
```

Algoritma

```
{ ambil input }  
input(v)  
input(t)  
{ hitung }  
s <- v * t { assignment mirip di R }  
{ keluarin }  
output(s, " m")
```

2.

Program Lingkaran

{ Menghitung keliling lingkaran }

Kamus

{ real di sini abstrak banget ternyata }

r, lingkaran: real

constant PI: real = 3.14159

Algoritma

input(r) *{ ambil r aja }*

lingkaran <- 2 * pi * r

output(lingkaran)

3.

Program Kelereng

{ Menghitung berapa kelereng yang harus dibayar }

Kamus

{ Jelek banget bahasa ini }

constant MERAH: integer = 10

constant HIJAU: integer = 15

constant KUNING: integer = 20

m, h, k: integer

total: integer

Algoritma

input(m)

input(h)

input(k)

total <- MERAH * m + HIJAU * h + KUNING * k

output(total)

4.

Program Point

{ Membuat point (?) soal aslinya kurang jelas sebenarnya }

Kamus

```
type Point:
  < absis : real;
    ordinat : real> { lexer & parser akan muntah ngeliat '<'
dan '>', mirip C++ }
P : Point
```

Algoritma

```
input(P) { asumsikan seperti ini karena high level sekali
ini }
output("(", P.absis, ",", P.ordinat, ")") { inilah kenapa
f-string/snprintf itu bagus }
```

Karena masih belum punya *linker* tersendiri, saya menggunakan gcc untuk *link* .o yang dioutput. Agar membuktikan bahwa bahasanya bisa di-run dan bukan sekadar konsep, saya kasih gambar saja. (Pakai *cat* di Arch Linux untuk membuktikan *source code*-nya memang itu dan ./out)

Jawaban Alkyl

1.

```
(1:16:44) faranaiki in ~/Git/alkyl at <main> [0]$ cat ./code/college/alpro_1.aky && ./alkyl code/college/alpro_1.aky && ./out
// sebenarnya ini harusnya trait tapi biarkan namanya masih prototipe, lagi pula ini showcase
extern int atoi(char*);

class Kecepatan {
    int kecepatan;
}

class Waktu {
    int waktu;
}

// has-a, compo bukan inheritance
open class Jarak has Waktu, Kecepatan {
    void out() {
        print "%d\n", this[Waktu].waktu * this[Kecepatan].kecepatan;
    }
}

int main() {
    let t = atoi(input());
    let v = atoi(input());
    Jarak s;
    s[Kecepatan].kecepatan = v;
    s[Waktu].waktu = t;
    s.out()
}
Compiled to out.o
Linking: gcc -g -O0 out.o -o out -no-pie
Linked to ./out
5
5
25
```

2.

```
(1:21:06) faranaiki in ~/Git/alkyl at <main> [0]$ cat ./code/college/alpro_2.aky && ./alkyl code/college/alpro_2.aky && ./out
define program as int main();

extern double atof(char*);

program {
    // anehnya ngga ada konstan di sini aakwoakwo, pake imut tapi belum diimplement
    let imut pi = 3.14159;
    let r = atof (input());
    let jumlah = 0 as double;
    for i in 2 {
        jumlah += pi * r;
    }
    print "%lf\n", jumlah;
}
Compiled to out.o
Linking: gcc -g -O0 out.o -o out -no-pie
Linked to ./out
5
15.707950
```

3.

```
(1:30:21) faranaiki in ~/Git/alkyl at <main> [0]$ cat ./code/college/alpro_3.aky && ./alkyl code/college/alpro_3.aky && ./out
define program as int main();
// Built-in Audio Analog Stereo
extern int atoi(char*);
Port: Speakers
program {
  // maaf, tapi array di commit ini sangat buggy dan nggak konsisten
  int *mhk = [10 as int, 15 as int, 20 as int];
  for i in 3 {
    int p = atoi(input());
    int t = mhk[i];
    mhk[i] = p * t;
  }
  let sum = 0;
  for i in 3 {
    sum += mhk[i];
  }
  print "%d\n", sum;
}
Compiled to out.o
Linking: gcc -g -O0 out.o -o out -no-pie
Linked to ./out
5
5
4
80
(1:30:23) faranaiki in ~/Git/alkyl at <main> [0]$
```

(sebenarnya ada kesalahan di sini karena +=, access pointer, dkk. masih belum terimplementasi sempurna. Mungkin 2 minggu lagi sudah full-feature, sebulan lagi sudah bisa self-hosted.

4.

Untuk nomor ini menggunakan bahasa pemrograman saya, saya izin pakai AI.

Soal 4

- Sebuah titik di atas bidang kartesian terdiri atas sebuah absis (x, bertipe real) dan ordinat (y, bertipe real)
- Buat program yang membaca sebuah data bertipe titik (misalnya P) dan menampilkannya ke layar dalam format: (x,y)

generate this code

reference syntax:
// sebenarnya ini harusnya trait tapi biarkan namanya masih prototipe, lagi pula ini showcase...

Analysis

Berdasarkan permintaan kamu dan referensi sintaksis bahasa **Alkyl** yang sedang dikembangkan, berikut adalah implementasi untuk membaca data bertipe titik dan menampilkannya dalam format `(x,y)`.

Karena Alkyl menggunakan pendekatan *composition* (menggunakan `has`) daripada

```

(1:30:23) faranaiki in ~/Git/alkyl at <main> [0]$ cat ./code/college/alpro_4.aky && ./alkyl code/college/alpro_4.aky && ./out
extern double atof(char*);

class Absis {
    double x;
}

class Ordinat {
    double y;
}

// Menggunakan pola 'has-a' (composition) sesuai referensi sintaksis
class Titik has Absis, Ordinat {
    void tampilkan() {
        // Mengakses komponen menggunakan sintaks indexing this[Component]
        print "(%f,%f)\n", this[Absis].x, this[Ordinat].y;
    }
}

int main() {
    // Membaca input dan konversi ke double
    let input_x = atof(input());
    let input_y = atof(input());

    // Inisialisasi objek Titik
    Titik P;

    // Memberikan nilai ke masing-masing komponen
    P[Absis].x = input_x;
    P[Ordinat].y = input_y;

    // Memanggil method untuk menampilkan hasil
    P.tampilkan();
}
Compiled to out.o
Linking: gcc -g -O0 out.o -o out -no-pie
Linked to ./out
5
4
(5.000000,4.000000)

```

(Perlu diperhatikan bahwa kode di atas hanya akan bisa compile jika dan hanya jika menggunakan commit `ce6282ccce2e92ba731fb0dbba1b3922f3ea79bd`, bisa dengan `git clone https://github.com/FaranAiki/alkyl` `git reset --hard ce6282ccce2e92ba731fb0dbba1b3922f3ea79bd`